

перезентации по лаборторной работе №7: Элементы криптографии. Однократное гаммирование

Эйвази Мани

06.октябрь.2026

Содержание

перезентации по лаборторной работе №7:	3
Цели и задачи работы	3
Цель лабораторной работы	3
Процесс выполнения лабораторной работы	4
Разработка приложения для шифрования и дешифрования в режиме однократного гаммирования . .	4
Реализация функций шифрования и дешифрования	6
Реализация режимов работы приложения	8
Запуск примера из лабораторной работы	10
Проверка дешифрования с неправильным ключом	11
Подбор ключа для целевого сообщения	12
Выводы по проделанной работе	13

Вывод	13
-----------------	----



презентации по лабораторной работе №7:

Элементы криптографии. Однократное гаммирование

Цели и задачи работы

Цель лабораторной работы

Освоить на практике применение режима однократного гаммирования.

Процесс выполнения лабораторной работы

Разработка приложения для шифрования и дешифрования в режиме одноразового гаммирования

- Разработано приложение `otp.py` на языке Python, реализующее режим одноразового гаммирования (One-Time Pad).
- Приложение поддерживает пять режимов работы:
 1. Шифрование (ключ + открытый текст → шифротекст).
 2. Дешифрование (ключ + шифротекст → открытый текст).
 3. Восстановление ключа (открытый текст + шифротекст → ключ).
 4. Подбор ключа для целевого сообщения.
 5. Запуск примера из лабораторной работы.

```

root@localhost:~/lab_07# cat crypto.txt
#!/usr/bin/env python3
"""
One-Time Pad (Gamma) Encryption Lab
"""

def xor_bytes(data: bytes, key: bytes) -> bytes:
    """XOR two byte sequences of equal length."""
    if len(data) != len(key):
        raise ValueError("Data and key must be the same length")
    return bytes(a ^ b for a, b in zip(data, key))

def hex_to_bytes(hex_str: str) -> bytes:
    return bytes.fromhex(hex_str.replace(" ", ""))

def bytes_to_hex(b: bytes) -> str:
    return " ".join(f"{x:02X}" for x in b)

def str_to_bytes(s: str, encoding='cp1251') -> bytes:
    """Convert string to bytes. CP1251 for Cyrillic text."""
    return s.encode(encoding)

def bytes_to_str(b: bytes, encoding='cp1251') -> str:
    return b.decode(encoding, errors='replace')

def main():
    print("=" * 50)
    print("ONE-TIME PAD (GAMMA) ENCRYPTION LAB")
    print("=" * 50)
    print("1. Encrypt (key + plaintext -> ciphertext)")
    print("2. Decrypt (key + ciphertext -> plaintext)")
    print("3. Recover key (plaintext + ciphertext -> key)")
    print("4. Find key for target message")
    print("5. Run example from lab")
    print("=" * 50)

    choice = input("Choose option (1-5): ").strip()

    if choice == "1":
        key_hex = input("Enter key (hex): ")
        plain_hex = input("Enter plaintext (hex): ")
        key = hex_to_bytes(key_hex)
        plain = hex_to_bytes(plain_hex)
        cipher = xor_bytes(plain, key)
        print("Ciphertext (hex):", bytes_to_hex(cipher))

```

Рис. 1 — Исходный код приложения otp.py (часть 1)

Реализация функций шифрования и дешифрования

- В приложении реализованы функции:
 - `xor_bytes(data, key)` — выполняет операцию XOR над двумя байтовыми последовательностями одинаковой длины.
 - `hex_to_bytes(hex_str)` — преобразует шестнадцатеричную строку в байты.
 - `bytes_to_hex(b)` — преобразует байты в шестнадцатеричную строку.
 - `str_to_bytes(s)` — преобразует строку в байты с использованием кодировки CP1251 (для кириллицы).
 - `bytes_to_str(b)` — преобразует байты в строку с использованием кодировки CP1251.

```

if choice == "1":
    key_hex = input("Enter key (hex): ")
    plain_hex = input("Enter plaintext (hex): ")
    key = hex_to_bytes(key_hex)
    plain = hex_to_bytes(plain_hex)
    cipher = xor_bytes(plain, key)
    print("Ciphertext (hex):", bytes_to_hex(cipher))

elif choice == "2":
    key_hex = input("Enter key (hex): ")
    cipher_hex = input("Enter ciphertext (hex): ")
    key = hex_to_bytes(key_hex)
    cipher = hex_to_bytes(cipher_hex)
    plain = xor_bytes(cipher, key)
    print("Plaintext (hex):", bytes_to_hex(plain))
    print("Plaintext (text):", bytes_to_str(plain))

elif choice == "3":
    plain_hex = input("Enter plaintext (hex): ")
    cipher_hex = input("Enter ciphertext (hex): ")
    plain = hex_to_bytes(plain_hex)
    cipher = hex_to_bytes(cipher_hex)
    key = xor_bytes(cipher, plain)
    print("Recovered key (hex):", bytes_to_hex(key))

elif choice == "4":
    cipher_hex = input("Enter ciphertext (hex): ")
    target = input("Enter target plaintext: ")
    cipher = hex_to_bytes(cipher_hex)
    target_bytes = str_to_bytes(target)
    if len(target_bytes) != len(cipher):
        print(f"Length mismatch: cipher={len(cipher)} bytes, target={len(target_bytes)} bytes")
        print("Adjust your target text to match ciphertext length.")
        return
    key = xor_bytes(cipher, target_bytes)
    print("Key to get target message:", bytes_to_hex(key))

elif choice == "5":

    key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54"
    plain_hex = "D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21 21"

    key = hex_to_bytes(key_hex)
    plain = hex_to_bytes(plain_hex)

    print("\n--- Example from Lab ---")

```

Рис. 2 — Исходный код приложения *отр.ру* (часть 2)

Реализация режимов работы приложения

- В функции `main()` реализованы все пять режимов работы приложения.
- Режим 1: шифрование — выполняет XOR открытого текста с ключом.
- Режим 2: дешифрование — выполняет XOR шифротекста с ключом.
- Режим 3: восстановление ключа — выполняет XOR шифротекста с открытым текстом.
- Режим 4: подбор ключа для целевого сообщения — выполняет XOR шифротекста с целевым открытым текстом.
- Режим 5: запуск примера из лабораторной работы.

```

cipher = hex_to_bytes(cipher_hex)
target_bytes = str_to_bytes(target)
if len(target_bytes) != len(cipher):
    print(f"Length mismatch: cipher={len(cipher)} bytes, target={len(target_bytes)} bytes")
    print("Adjust your target text to match ciphertext length.")
    return
key = xor_bytes(cipher, target_bytes)
print("Key to get target message:", bytes_to_hex(key))

elif choice == "5":

    key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54"
    plain_hex = "D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21 21"

    key = hex_to_bytes(key_hex)
    plain = hex_to_bytes(plain_hex)

    print("\n--- Example from Lab ---")
    print("Key:", key_hex)
    print("Plaintext (hex):", plain_hex)
    print("Plaintext (text):", bytes_to_str(plain))

    cipher = xor_bytes(plain, key)
    print("Ciphertext (hex):", bytes_to_hex(cipher))

    recovered_key = xor_bytes(cipher, plain)
    print("Recovered key (hex):", bytes_to_hex(recovered_key))
    print("Key match:", bytes_to_hex(recovered_key) == key_hex)

    wrong_key_hex = "05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 55 F4 D3 07 BB BC 54"
    wrong_key = hex_to_bytes(wrong_key_hex)
    wrong_plain = xor_bytes(cipher, wrong_key)
    print("\n--- Wrong Key Decryption ---")
    print("Wrong key:", wrong_key_hex)
    print("Result (hex):", bytes_to_hex(wrong_plain))
    print("Result (text):", bytes_to_str(wrong_plain))

else:
    print("Invalid choice")

if __name__ == "__main__":
    main()

```

Рис. 3 — Исходный код приложения otr.py (часть 3)

Запуск примера из лабораторной работы

- Выполнена команда `chmod +x otp.py` для предоставления файлу прав на исполнение.
- Выполнена команда `./otp.py` для запуска приложения.
- Выбран режим 5 — «Run example from lab».
- Приложение вывело:
 - Key: 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
 - Plaintext (hex): D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21 21
 - Plaintext (text): - !!
 - Ciphertext (hex): DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75
 - Recovered key (hex): 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
 - Key match: True

```
root@localhost:~/lab_07# chmod +x otp.py
root@localhost:~/lab_07# ./otp.py
=====
ONE-TIME PAD (GAMMA) ENCRYPTION LAB
=====
1. Encrypt (key + plaintext -> ciphertext)
2. Decrypt (key + ciphertext -> plaintext)
3. Recover key (plaintext + ciphertext -> key)
4. Find key for target message
5. Run example from lab
=====
Choose option (1-5): 5

--- Example from Lab ---
Key: 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
Plaintext (hex): D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21 21
Plaintext (text): Ширлиц - Вы Герой!!
Ciphertext (hex): DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75
Recovered key (hex): 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54
Key match: True

--- Wrong Key Decryption ---
Wrong key: 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 55 F4 D3 07 BB BC 54
Result (hex): D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C1 EE EB E2 E0 ED 21
Result (text): Ширлиц - Вы Болван!
```

Рис. 4 — Запуск примера из лабораторной работы

Проверка дешифрования с неправильным ключом

- В примере из лабораторной работы также продемонстрировано дешифрование с неправильным ключом:
 - Wrong key: 05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 55 F4 D3 07 BB BC 54
 - Result (hex): D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C1 EE EB E2 E0 ED 21
 - Result (text): - !
- Это демонстрирует, что при использовании неправильного ключа получается искажённый открытый текст.

```
root@localhost:~/lab_07# cat cypher
ciphertext: D0 FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 58 51 75
target plaintext: С Новым Годом, друзья
root@localhost:~/lab_07# ./otp.py
=====
ONE-TIME PAD (GAMMA) ENCRYPTION LAB
=====
1. Encrypt (key + plaintext -> ciphertext)
2. Decrypt (key + ciphertext -> plaintext)
3. Recover key (plaintext + ciphertext -> key)
4. Find key for target message
5. Run example from lab
=====
Choose option (1-5): 4
Enter ciphertext (hex): D0 FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 58 51 75
Enter target plaintext: С Новым Годом, друзья
Key to get target message: 0C DE 32 61 07 5D 2D D2 7A DE 2F 3B EE B8 3A DC 15 A8 B6 8A
```

Рис. 5 — Дешифрование с неправильным ключом

Подбор ключа для целевого сообщения

- Выбран режим 4 — «Find key for target message».
 - Введён шифротекст: DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75.
 - Введён целевой открытый текст: , .
 - Приложение вычислило ключ: 0C DE 32 61 07 5D 20 D2 7A DE 2F 3B EE BB 3A DC 15 A8 B6 8A.
 - Полученный ключ позволяет преобразовать исходный шифротекст в целевое сообщение «С Новым Годом, друзья».
- Используется операция XOR (сложение по модулю 2).
 - Особенности: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. Двойное применение XOR с одним и тем же ключом восстанавливает исходные данные.
 - $C_i = P_i \oplus K_i$, где C_i — шифротекст, P_i — открытый текст, K_i — ключ.

Рис. 6 — Подбор ключа для целевого сообщения

Выводы по проделанной работе

Вывод

В ходе лабораторной работы было освоено на практике применение режима однократного гаммирования. Разработано приложение `otr.py`, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования, а также восстанавливать ключ по известным открытому тексту и шифротексту и подбирать ключ для получения заданного целевого сообщения.

Продемонстрировано, что при использовании правильного ключа исходный текст восстанавливается без искажений, а при использовании неправильного ключа получается искажённый текст. Показано, что знание шифротекста и целевого открытого текста позволяет вычислить ключ, с помощью которого можно преобразовать шифротекст в любое сообщение той же длины.

Полученные результаты согласуются с целью работы.